

MPI-Based Distributed Inference for Graph Neural Network Surrogates on Finite Element Meshes

Andréane Maure

May 2026

Abstract

Finite Element Analysis (FEA) is widely used to predict the mechanical response of engineering structures, but repeated high-fidelity simulations can become computationally expensive. Graph Neural Networks (GNNs) provide a promising surrogate modeling approach by learning nodal displacement and stress fields directly from mesh-based simulation data. However, for large-scale finite element meshes, full-graph inference on a single process may become limited by memory and computational cost.

This work develops a proof-of-concept distributed inference pipeline for a trained GNN surrogate using MPI and METIS graph partitioning. The finite element graph is decomposed into balanced subdomains, where each MPI rank owns a subset of nodes and stores ghost nodes required for cross-partition message passing. Since the GNN contains multiple message-passing layers, hidden node embeddings are synchronized between neighboring ranks through layer-wise halo exchange. After inference, owned-node predictions are gathered and reconstructed into a global output field.

The MPI implementation is validated by comparing the reconstructed distributed prediction with the original serial full-graph inference. For all tested MPI configurations and simulation cases, the distributed result exactly reproduces the serial prediction. Scaling experiments show limited speedup on the current small graphs, as communication overhead becomes comparable to local computation. Nevertheless, the implementation validates the core communication pattern required for future distributed GNN inference on larger industrial FEM meshes.

1 Introduction

Finite Element Analysis (FEA) is a standard numerical tool for predicting the mechanical response of engineering structures by computing displacement, strain, and stress fields on a discretized domain [1]. Although FEA provides high-fidelity predictions, repeated simulations can become computationally expensive in design exploration, optimization, or real-time engineering workflows. Surrogate modeling aims to reduce this cost by replacing repeated high-fidelity simulations with faster predictive models [2].

Graph Neural Networks (GNNs) are well suited for surrogate modeling on mesh-based physical systems because a finite element mesh can naturally be represented as a graph, with mesh nodes as graph nodes and element connectivity defining graph edges. In message-passing GNNs, nodal representations are updated by aggregating information from neighboring nodes [3]. Related graph-network architectures have been successfully applied to physical simulation and mesh-based learning tasks [4]. In the main project, a trained GNN surrogate predicts nodal displacement and stress fields from finite element graphs generated with Abaqus.

In the standard inference pipeline, the trained GNN is evaluated on the full graph using a single process. This is sufficient for the current dataset, where each graph contains only a few thousand nodes, but it does not directly address the scalability challenges that may arise for larger industrial finite element meshes. The objective of this work is therefore not to retrain the model or improve its physical accuracy, but to reformulate full-graph GNN inference as a distributed computing problem while preserving the original serial prediction.

To achieve this, the finite element graph is partitioned into subdomains using METIS graph partitioning [5]. Each MPI rank owns one partition and stores additional ghost nodes required for message passing across partition boundaries. Since the GNN contains multiple message-passing layers, hidden node embeddings must be synchronized between neighboring ranks through layer-wise halo exchange. This communication pattern follows the domain decomposition viewpoint commonly used in parallel numerical simulations [6] and is implemented using MPI communication [7].

The main contribution of this report is a proof-of-concept MPI pipeline for distributed GNN inference on METIS-partitioned finite element graphs. The implementation includes graph partitioning, local subgraph construction, owned and ghost node identification, halo exchange, global prediction reconstruction, and validation against serial full-graph inference. The results show that the distributed MPI implementation exactly reproduces the serial GNN prediction for all tested cases, while the current scaling remains limited by communication overhead on small graphs.

2 Background

2.1 FEM Meshes as Graphs

Finite element simulations are naturally defined on discretized domains, where the geometry is represented by a mesh composed of nodes and elements. In solid mechanics, these meshes are used to approximate displacement, strain, and stress fields over the computational domain [1]. In this project, each Abaqus simulation corresponds to a two-dimensional cantilever beam discretized with quadrilateral finite elements and converted into a graph representation for graph-based learning.

A graph is defined as

$$G = (V, E),$$

where V is the set of graph nodes and E is the set of graph edges. Each finite element node is represented as a graph node, and the graph connectivity is derived from the finite element connectivity. Edges are created between neighboring nodes of each quadrilateral element, duplicate edges are removed, and the graph is made bidirectional to allow information exchange in both directions during message passing.

Node features encode normalized coordinates, boundary conditions, loading information, global simulation parameters, and geometric information. Edge attributes encode local geometric relations between connected nodes, such as relative coordinate differences and edge length. The target quantities are nodal displacement components and normal stress,

$$\mathbf{y}_i = \begin{bmatrix} u_{x,i} & u_{y,i} & \sigma_{xx,i} \end{bmatrix}.$$

This representation preserves the local neighborhood structure of the finite element mesh while providing a format suitable for graph-based learning on mesh-based physical systems [8, 4].

2.2 GNN Surrogate Model

The surrogate model used in this project is a message-passing Graph Neural Network trained to predict nodal displacement and stress fields from finite element graphs. GNNs are well suited to mesh-based physical systems because they operate directly on graph-structured data and update node representations using information from neighboring nodes [3, 9].

The model follows an encoder–processor–decoder architecture, a common structure in graph-based physical simulation models [8, 4]. The encoder maps raw node features into a latent representation. The processor applies multiple message-passing layers to update node embeddings along graph edges. The decoder then maps the final node embeddings to the predicted physical fields,

$$\hat{\mathbf{y}}_i = \begin{bmatrix} \hat{u}_{x,i} & \hat{u}_{y,i} & \hat{\sigma}_{xx,i} \end{bmatrix}.$$

The model is trained in a supervised setting using finite element simulation data as ground truth. It learns a mapping from mesh geometry, boundary conditions, and loads to nodal displacement and

stress fields. Once trained, the model can perform inference faster than running a new high-fidelity finite element simulation. However, in the standard implementation, inference is performed on the full graph using a single process, which motivates the distributed inference strategy studied in this report.

2.3 Message Passing and Data Dependencies

The central operation in the GNN is message passing. At each layer, every node updates its hidden representation by aggregating information from neighboring nodes [3]. A generic message-passing update can be written as

$$\mathbf{m}_i^{(k)} = \text{AGG}_{j \in \mathcal{N}(i)} \phi^{(k)} \left(\mathbf{h}_i^{(k)}, \mathbf{h}_j^{(k)}, \mathbf{e}_{ij} \right),$$

$$\mathbf{h}_i^{(k+1)} = \psi^{(k)} \left(\mathbf{h}_i^{(k)}, \mathbf{m}_i^{(k)} \right),$$

where $\mathbf{h}_i^{(k)}$ is the hidden representation of node i at layer k , $\mathcal{N}(i)$ is the set of neighboring nodes, $\mathbf{e}_{ij}$ is the edge attribute between nodes i and j , and $\phi^{(k)}$ and $\psi^{(k)}$ are neural network functions.

This operation creates a direct data dependency between neighboring nodes. In serial full-graph inference, all node embeddings are available in memory, so these dependencies are handled implicitly. In distributed inference, the graph is split across multiple MPI ranks. If two connected nodes are assigned to different ranks, the hidden embedding of the remote node must be communicated before the local update can reproduce the full-graph computation.

This dependency is the motivation for using ghost nodes and halo exchange. Ghost nodes store local copies of neighboring nodes owned by other ranks, and their embeddings are synchronized during message passing. Without this communication, each rank would only propagate information inside its local partition, and the distributed prediction would differ from the original serial GNN output.

3 Distributed Inference Formulation

In the serial baseline, the trained GNN is evaluated on the complete finite element graph using a single process. The full graph is denoted by

$$G = (V, E),$$

where V contains all mesh nodes and E contains the graph edges derived from the finite element connectivity. The model computes predictions for all nodes as

$$\hat{\mathbf{Y}}_{\text{serial}} = f_{\theta}(G),$$

where f_θ is the trained GNN and $\hat{\mathbf{Y}}_{\text{serial}}$ contains the predicted displacement and stress fields. In this setting, all node embeddings are stored in a single memory space, so message-passing dependencies between neighboring nodes are resolved without explicit communication.

The objective of distributed inference is to reproduce the same full-graph prediction when the graph is split across P MPI ranks. The node set is partitioned into subdomains,

$$V = V_1 \cup V_2 \cup \dots \cup V_P,$$

where each subset V_p is assigned to one rank. The partitioning is performed using METIS, which aims to balance the number of nodes per partition while reducing the number of cut edges between partitions [5]. Each rank performs local computation on its assigned subdomain, but cut edges introduce dependencies between nodes owned by different ranks.

To handle these dependencies, each rank stores both owned nodes and ghost nodes. Owned nodes are assigned to the rank by the graph partitioning, while ghost nodes are local copies of neighboring nodes owned by other ranks. This follows the domain decomposition viewpoint used in parallel numerical simulations, where each subdomain stores local data and halo information from neighboring subdomains [6]. For rank p , the local node set is

$$V_p^{\text{local}} = V_p^{\text{owned}} \cup V_p^{\text{ghost}}.$$

During message passing, the rank computes updates for its owned nodes using both owned and ghost nodes. Since the GNN contains multiple message-passing layers, ghost-node embeddings must be synchronized layer by layer. This synchronization is performed through halo exchange: boundary node embeddings are exchanged between neighboring ranks so that each local graph contains the same neighbor information as in the serial full-graph computation.

After the final layer, each rank decodes predictions only for its owned nodes. These owned-node predictions are gathered and reconstructed in the original global node ordering:

$$\hat{\mathbf{Y}}_{\text{MPI}} = \text{Gather} \left(\hat{\mathbf{Y}}_1^{\text{owned}}, \hat{\mathbf{Y}}_2^{\text{owned}}, \dots, \hat{\mathbf{Y}}_P^{\text{owned}} \right).$$

The correctness criterion is that the reconstructed MPI prediction matches the serial full-graph prediction,

$$\hat{\mathbf{Y}}_{\text{MPI}} \approx \hat{\mathbf{Y}}_{\text{serial}}.$$

In the experiments, this comparison is performed on normalized model outputs to verify that the distributed computation reproduces the original GNN inference before any denormalization or post-processing.

4 MPI/METIS Method

4.1 Graph Partitioning and Local Subgraphs

The distributed inference pipeline starts by partitioning the finite element graph into P subdomains, where P is the number of MPI ranks. This partitioning is performed with METIS, which aims to balance the number of nodes per partition while reducing the number of cut edges between partitions [5]. In distributed GNN inference, balanced partitions improve workload distribution, while fewer cut edges reduce the amount of halo communication.

Given the full graph

$$G = (V, E),$$

METIS assigns each node $i \in V$ to a partition

$$p(i) \in \{0, 1, \dots, P - 1\}.$$

Each MPI rank p owns the node set

$$V_p^{\text{owned}} = \{i \in V \mid p(i) = p\}.$$

Edges connecting nodes from different partitions are cut edges and define the communication pattern between neighboring ranks.

After partitioning, each rank builds a local graph containing its owned nodes and the ghost nodes required for message passing across cut edges:

$$V_p^{\text{local}} = V_p^{\text{owned}} \cup V_p^{\text{ghost}}.$$

A mapping between global and local node indices is constructed so that the GNN can operate on compact local tensors while still allowing the final predictions to be reconstructed in the original global ordering. The local graph includes the node features, edge connectivity, and edge attributes needed to compute messages for owned nodes. Ghost nodes are used only to provide neighboring embeddings during message passing; final predictions are kept only for owned nodes.

4.2 Layer-wise Halo Exchange

The main challenge is that message passing creates dependencies across partition boundaries. At each GNN layer, a node update depends on the hidden embeddings of neighboring nodes. Therefore, if a neighbor is owned by another rank, its embedding must be available locally through a ghost node.

Since the trained GNN contains multiple message-passing layers, ghost-node embeddings must be synchronized layer by layer. Each rank exchanges boundary node embeddings with its neighboring ranks using MPI communication through `mpi4py` [7, 10]. This follows the same halo-exchange

idea used in domain decomposition methods for parallel numerical simulations [6].

For each message-passing layer, the local computation follows:

exchange boundary embeddings $\rightarrow$ update ghost embeddings $\rightarrow$ apply local message passing.

In the implementation, the halo exchange is performed after each message-passing layer to refresh ghost-node embeddings before the next layer. No exchange is required before the first layer because ghost-node input features are already available locally and are encoded with the same shared encoder. This ensures that each rank has the same neighbor information that would be available in the serial full-graph computation.

A simplified version of the algorithm is:

```
For each MPI rank p:
  Load owned nodes and ghost nodes
  Build local graph tensors
  Encode local node features

  For each message-passing layer k:
    Exchange boundary embeddings with neighbor ranks
    Update ghost-node embeddings
    Apply message passing on the local graph

  Decode predictions for owned nodes only
  Send owned predictions to rank 0
```

4.3 Global Prediction Reconstruction

After the final message-passing layer, each rank decodes predictions for its local nodes but retains only the predictions associated with owned nodes. Ghost-node predictions are discarded because those nodes are owned by other ranks. Each rank sends its owned-node predictions and corresponding global node indices to rank 0.

Rank 0 reconstructs the full prediction tensor by placing each owned-node prediction back into the original global node ordering. The reconstructed MPI prediction is then compared with the serial full-graph prediction. This comparison verifies that the partitioning, halo exchange, and reconstruction steps preserve the original GNN inference.

5 Implementation

The implementation is available in the project GitHub repository:¹ The HPC-specific part of the project is contained in the `hpc/` directory. This folder is separate from the main GNN training

¹<https://github.com/am6823/gnn-fea-surrogate.git>

and evaluation pipeline and contains the scripts used for METIS partitioning, halo construction, MPI inference, serial reference inference, and scaling analysis.

The graph data structures and trained GNN model are reused from the main PyTorch Geometric pipeline [11], while the distributed execution is implemented using MPI through `mpi4py` [10]. This structure makes it possible to keep the original full-graph GNN model unchanged while adding an HPC layer for partitioned inference.

The implementation follows a modular workflow. First, `partition_metis.py` partitions the selected test graph into the desired number of subdomains. Then, `build_halo_data.py` identifies the owned nodes, ghost nodes, boundary nodes, and neighboring ranks for each partition. The script `serial_reference_inference.py` computes the full-graph prediction used as a numerical reference, while `mpi_partitioned_inference.py` performs distributed inference with MPI halo exchange. Finally, `compare_serial_mpi.py` and `plot_scaling.py` are used to compare MPI predictions with the serial reference and generate scaling results.

The complete command-line workflow is documented in the repository README. In this report, the implementation is evaluated through correctness metrics comparing MPI and serial predictions, and through performance metrics including runtime, speedup, efficiency, and communication time.

6 Experimental Setup

6.1 Test Cases

The distributed inference pipeline is evaluated on three test simulations from the held-out test set. The main validation case is `sim_id = 124`, which is used for both correctness validation and scaling analysis. Two additional simulations are used to verify that the MPI implementation is not specific to a single graph.

Table 1: Test cases used for MPI validation.

<code>sim_id</code>	<code>sim_index</code>	Nodes	Edges
124	0	4623	18044
144	3	6231	24460
52	8	2613	10024

For the main case, MPI inference is tested with 1, 2, 4, and 8 ranks. The additional cases are tested with 4 and 8 ranks. All experiments use the same trained GNN model and the same graph data as the serial inference pipeline.

6.2 Correctness Metrics

The main goal of the MPI implementation is to reproduce the original serial full-graph prediction. Therefore, correctness is evaluated by comparing the reconstructed MPI prediction with the

serial prediction:

$$\hat{\mathbf{Y}}_{\text{MPI}} \quad \text{vs.} \quad \hat{\mathbf{Y}}_{\text{serial}}.$$

This comparison is performed on normalized model outputs, before denormalization. This avoids introducing differences from post-processing and directly tests whether the distributed computation reproduces the original GNN inference.

The following metrics are used:

$$\begin{aligned} \text{max abs diff} &= \max_i \left| \hat{\mathbf{Y}}_{\text{MPI},i} - \hat{\mathbf{Y}}_{\text{serial},i} \right|, \\ \text{mean abs diff} &= \frac{1}{N} \sum_i \left| \hat{\mathbf{Y}}_{\text{MPI},i} - \hat{\mathbf{Y}}_{\text{serial},i} \right|, \end{aligned}$$

and the global relative L^2 difference,

$$\text{Rel } L^2 = \frac{\left\| \hat{\mathbf{Y}}_{\text{MPI}} - \hat{\mathbf{Y}}_{\text{serial}} \right\|_2}{\left\| \hat{\mathbf{Y}}_{\text{serial}} \right\|_2}.$$

A relative L^2 difference close to zero indicates that the MPI prediction is numerically equivalent to the serial full-graph prediction.

6.3 Performance Metrics

The scaling behavior is evaluated using runtime, speedup, parallel efficiency, and communication time. Let T_p denote the total inference runtime using p MPI ranks. The speedup is defined as

$$S_p = \frac{T_1}{T_p},$$

where T_1 is the runtime of the MPI implementation using one rank. The parallel efficiency is defined as

$$E_p = \frac{S_p}{p}.$$

Communication time measures the time spent exchanging boundary embeddings between MPI ranks during halo exchange. This metric is important because the current graphs are relatively small, so communication overhead can become comparable to local computation. The performance results are therefore interpreted as a strong-scaling experiment on small FEM graphs, rather than as a production-scale benchmark.

7 Results

7.1 METIS Partitioning Results

The main validation case corresponds to `sim_id = 124`, which contains 4623 nodes and 18044 edges. The graph was partitioned using METIS for 2, 4, and 8 partitions. The resulting partition statistics are shown in Table 2.

Table 2: METIS partition statistics for `sim_id = 124`.

Partitions	Min size	Max size	Cut edges
2	2311	2312	25
4	1155	1156	76
8	577	578	193

The partitions are well balanced for all tested partition counts. The minimum and maximum partition sizes differ by at most one node, which indicates that METIS produces an almost perfectly balanced workload for this graph. As expected, the number of cut edges increases with the number of partitions. This is because more subdomains introduce more partition boundaries, which increases the number of cross-partition dependencies that must be handled through halo exchange.

The 8-partition case is visualized in Fig. 1. The partitioning follows the beam-like structure of the finite element mesh, with each color corresponding to one MPI rank.

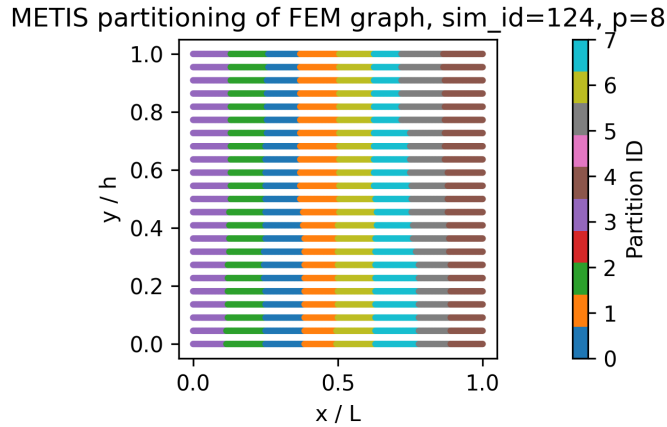


Figure 1: METIS partitioning of the finite element graph for `sim_id = 124` using 8 partitions. Each color corresponds to one MPI rank.

7.2 Halo Data Analysis

For the 8-partition case, the halo construction identifies the owned nodes, ghost nodes, boundary nodes, local nodes, and neighboring ranks associated with each MPI rank. The results are shown in Table 3.

Table 3: Halo data for `sim_id` = 124 with 8 partitions.

Rank	Owned	Ghost	Boundary	Local	Neighbors
0	577	47	47	624	[1, 2]
1	578	47	47	625	[0, 6]
2	578	46	46	624	[0, 3]
3	578	23	23	601	[2]
4	578	24	24	602	[5]
5	578	54	54	632	[4, 7]
6	578	46	46	624	[1, 7]
7	578	53	53	631	[5, 6]

The total number of ghost nodes counted with duplicates is 340, and the maximum number of ghost nodes on a single rank is 54. Since each rank owns approximately 578 nodes, the largest ghost-to-owned ratio is approximately

$$\frac{54}{578} \approx 9.3\%.$$

This indicates that the halo overhead remains moderate for this graph. The number of ghost nodes varies between ranks depending on the local partition boundary length and the number of neighboring partitions.

7.3 Correctness Against Serial Inference

The main correctness criterion is that the reconstructed MPI prediction must match the original serial full-graph prediction. The comparison is performed on normalized model outputs before denormalization:

$$\hat{\mathbf{Y}}_{\text{MPI}} \text{ vs. } \hat{\mathbf{Y}}_{\text{serial}}.$$

For `sim_id` = 124, the MPI implementation exactly reproduces the serial prediction for all tested rank counts, as shown in Table 4.

Table 4: Correctness comparison between MPI and serial inference for `sim_id` = 124.

MPI ranks	Max abs diff	Mean abs diff	Rel. L^2 diff
1	0.0	0.0	0.0
2	0.0	0.0	0.0
4	0.0	0.0	0.0
8	0.0	0.0	0.0

This result validates the distributed inference pipeline. It confirms that the graph partitioning is consistent, that ghost nodes are correctly identified, that halo exchange synchronizes the required hidden embeddings, and that the final owned-node predictions are reconstructed in the correct global ordering.

Since the MPI prediction is identical to the serial prediction, the physical accuracy with respect

to the FEM ground truth is preserved. The purpose of the MPI implementation is therefore not to improve the trained model, but to reproduce its full-graph inference under distributed execution.

7.4 Runtime and Scaling Results

The runtime and scaling results for `sim_id = 124` are shown in Table 5. The 1-rank MPI runtime is used as the baseline for computing speedup and efficiency.

Table 5: Runtime and scaling results for `sim_id = 124`.

Ranks	Runtime (s)	Speedup	Efficiency	Comm. time (s)
1	0.061752	1.00	1.00	0.000002
2	0.043924	1.41	0.70	0.001572
4	0.064659	0.96	0.24	0.003431
8	0.049655	1.24	0.16	0.007202

The 2-rank case achieves the best speedup, with a speedup of 1.41 and a parallel efficiency of 0.70. For 4 and 8 ranks, the scaling is limited. This behavior is expected because the graph contains only a few thousand nodes, so the local computation per rank is small. As the number of ranks increases, communication overhead becomes comparable to the cost of local GNN computation.

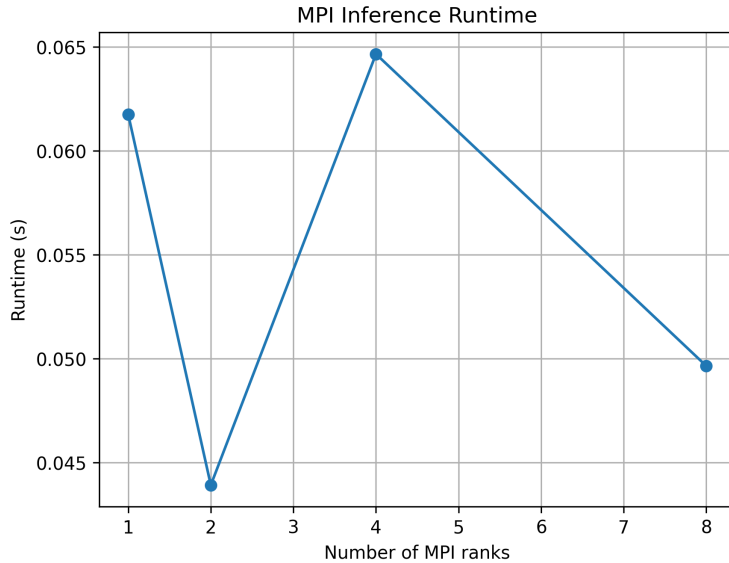


Figure 2: Total MPI inference runtime as a function of the number of ranks for `sim_id = 124`.

Fig. 2 shows that runtime does not decrease monotonically with the number of MPI ranks. The 2-rank case is faster than the 1-rank case, but additional ranks do not lead to consistent runtime reduction because the graph is too small for the extra parallelism to dominate the MPI overhead.

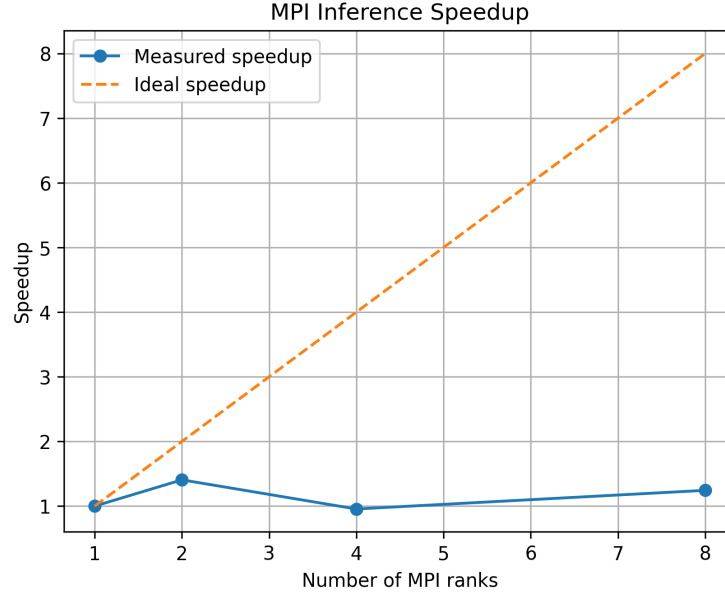


Figure 3: MPI speedup as a function of the number of ranks for `sim_id = 124`.

Fig. 3 shows that the measured speedup remains far from ideal linear scaling as the number of ranks increases. The best speedup is obtained with 2 MPI ranks, while the 4-rank and 8-rank cases are limited by communication and synchronization overhead. This confirms that the current graph is too small to fully benefit from additional parallelism.

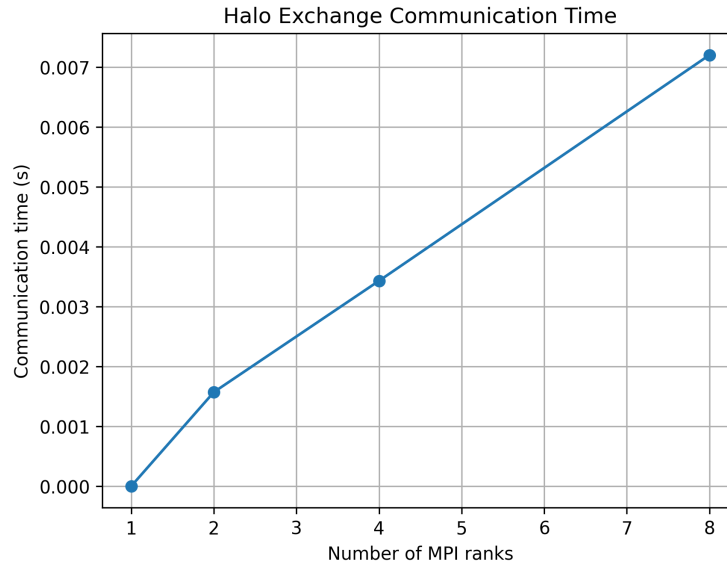


Figure 4: Halo exchange communication time as a function of the number of ranks for `sim_id = 124`.

Fig. 4 shows that communication time increases with the number of ranks, from approximately 1.6×10^{-3} seconds for 2 ranks to 7.2×10^{-3} seconds for 8 ranks. This reflects the increase in

partition boundaries and halo exchanges as the graph is divided into more subdomains.

Overall, these results should be interpreted as a strong-scaling experiment on a small graph. The limited speedup does not indicate an incorrect implementation. Instead, it shows that, for small workloads, MPI overhead can dominate before significant parallel acceleration is observed.

7.5 Additional Validation Cases

The distributed inference pipeline was also tested on two additional simulations to verify that the implementation is not specific to `sim_id = 124`. The additional cases are summarized in Table 6.

Table 6: Additional MPI validation cases.

<code>sim_id</code>	Nodes	Edges	Ranks	Rel. L^2 diff
144	6231	24460	4	0.0
144	6231	24460	8	0.0
52	2613	10024	4	0.0
52	2613	10024	8	0.0

For both additional simulations, the reconstructed MPI prediction exactly matches the serial full-graph prediction. This confirms that the method generalizes across graphs with different numbers of nodes, edges, and partition structures.

The larger case, `sim_id = 144`, contains 6231 nodes and 24460 edges, while the smaller case, `sim_id = 52`, contains 2613 nodes and 10024 edges. In both cases, correctness is preserved for 4 and 8 MPI ranks. This supports the main conclusion that the implemented partitioning, halo exchange, and reconstruction strategy is consistent across multiple finite element graphs.

8 Discussion

8.1 Correctness of Distributed Message Passing

The main result of this work is that the MPI implementation exactly reproduces the serial full-graph GNN inference for all tested rank counts and validation cases. This confirms that the distributed message-passing strategy is consistent with the original GNN computation. In particular, the results validate three key steps of the pipeline: the construction of local graphs with owned and ghost nodes, the layer-wise halo exchange of hidden embeddings, and the reconstruction of owned-node predictions into the original global node ordering.

This result is important because graph partitioning alone is not sufficient to reproduce full-graph inference. Without halo exchange, nodes located near partition boundaries would only receive messages from nodes inside their local partition, leading to a different computation from the serial GNN. The zero relative L^2 difference between MPI and serial predictions shows that the implemented halo exchange preserves the required cross-partition dependencies at every message-passing layer.

8.2 Interpretation of Scaling Results

The scaling results show a moderate speedup for 2 MPI ranks, but limited scalability for larger rank counts. This behavior is expected for the current test graphs, which contain only a few thousand nodes. In this regime, the amount of local computation assigned to each rank becomes small, while MPI communication, synchronization, and data gathering introduce overhead.

As the number of ranks increases, the number of partition boundaries and cut edges also increases. This leads to more ghost nodes and more halo exchanges between neighboring ranks. Consequently, communication time increases with the number of ranks. For small graphs, this communication overhead can become comparable to, or larger than, the local GNN computation time. Therefore, the scaling results should be interpreted as a proof-of-concept strong-scaling experiment rather than as a production-level performance benchmark.

8.3 Relevance for Larger FEM Graphs

Although the current graphs are relatively small, the implemented approach introduces the core communication pattern required for distributed GNN inference on larger finite element meshes. For larger 2D or 3D industrial meshes, full-graph inference may become memory- or compute-limited on a single process. In that context, distributing the graph across multiple ranks could become necessary.

The present implementation already includes the essential ingredients for this larger-scale setting: graph partitioning, owned and ghost nodes, halo exchange, and global prediction reconstruction. On larger graphs, the ratio between computation and communication is expected to become more favorable, because each rank would process a larger local subgraph. Therefore, the main contribution of this work is not the speedup obtained on the current small graphs, but the validation of a correctness-preserving distributed inference strategy.

9 Limitations and Future Work

This work has several limitations. First, the experiments are performed on relatively small 2D finite element graphs, so strong scalability is limited by communication overhead. Second, the current implementation focuses only on inference and does not address distributed training. Third, the communication is implemented for CPU tensors using `mpi4py`, without GPU-aware MPI or overlap between communication and computation. Finally, the method has not yet been tested on large 3D industrial meshes, where distributed execution would be more clearly motivated.

Future work should therefore focus on evaluating the method on larger and more complex finite element graphs, especially 3D meshes with significantly higher node and edge counts. Communication could also be optimized by reducing unnecessary halo exchanges, overlapping communication with local computation, or using GPU-aware MPI. Another natural extension would be to apply similar partitioning and halo-exchange ideas to distributed GNN training. More broadly, this

pipeline could be integrated into a larger simulation workflow where trained GNN surrogates are deployed for fast prediction on large-scale engineering meshes.

10 Conclusion

This report presented a proof-of-concept MPI pipeline for distributed GNN inference on METIS-partitioned finite element graphs. Starting from a trained GNN surrogate model, the full FEM graph was decomposed into subdomains, with each MPI rank owning a subset of nodes and storing ghost nodes required for cross-partition message passing. Layer-wise halo exchange was used to synchronize hidden node embeddings, and owned-node predictions were gathered to reconstruct the global output field.

The main result is that the distributed MPI implementation exactly reproduces the serial full-graph inference for all tested rank counts and validation cases. This confirms that the partitioning, ghost-node construction, halo exchange, and global reconstruction steps preserve the original GNN computation. Therefore, the MPI pipeline does not change the physical prediction of the trained surrogate model; it only changes how inference is executed computationally.

The scaling results show limited speedup on the current small graphs, with communication overhead becoming significant as the number of MPI ranks increases. This behavior is expected because the test graphs contain only a few thousand nodes. Nevertheless, the implementation validates the core communication pattern needed for future distributed inference on larger FEM meshes. This work therefore represents a first step toward scalable deployment of graph-based surrogate models for large-scale engineering simulations.

References

- [1] T. J. R. Hughes. *The Finite Element Method: Linear Static and Dynamic Finite Element Analysis*. Dover Publications, 2000.
- [2] A. I. J. Forrester, A. Sóbester, and A. J. Keane. *Engineering Design via Surrogate Modelling: A Practical Guide*. Wiley, 2008.
- [3] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, pages 1263–1272, 2017.
- [4] T. Pfaff, M. Fortunato, A. Sanchez-Gonzalez, and P. W. Battaglia. Learning mesh-based simulation with graph networks. In *International Conference on Learning Representations*, 2021.
- [5] G. Karypis and V. Kumar. Multilevel k-way partitioning scheme for irregular graphs. *Journal of Parallel and Distributed Computing*, 48(1):96–129, 1998.

- [6] A. Toselli and O. Widlund. *Domain Decomposition Methods: Algorithms and Theory*. Springer, 2005.
- [7] W. Gropp, E. Lusk, and A. Skjellum. *Using MPI: Portable Parallel Programming with the Message-Passing Interface*. MIT Press, 2nd edition, 1999.
- [8] A. Sanchez-Gonzalez, J. Godwin, T. Pfaff, R. Ying, J. Leskovec, and P. W. Battaglia. Learning to simulate complex physics with graph networks. In *International Conference on Machine Learning*, pages 8459–8468, 2020.
- [9] P. W. Battaglia, J. B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, V. Zambaldi, M. Malinowski, A. Tacchetti, D. Raposo, A. Santoro, R. Faulkner, and others. Relational inductive biases, deep learning, and graph networks. *arXiv preprint*, arXiv:1806.01261, 2018.
- [10] L. D. Dalcin, P. A. Kler, R. R. Paz, and A. Cosimo. Parallel distributed computing using Python. *Advances in Water Resources*, 34(9):1124–1139, 2011.
- [11] M. Fey and J. E. Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.